AgentFly: Extensible and Scalable Reinforcement Learning for LLM Agents

Renxi Wang[1] **Bilal Elbouardi**[1] **Rifo Ahmad Genadi**[1] **Yongxin Wang**[1]
Shu Wang[2] **Fajri Koto**[1] **Zhengzhong Liu**[1] **Timothy Baldwin** [1]
Haonan Li [1]

[1]Mohamed bin Zayed University of Artificial Intelligence [2]University of California, Los Angeles

{renxi.wang,haonan.li}@mbzuai.ac.ae

Methods to build LLM agents have evolved from prompt engineering and supervised finetuning to agentic reinforcement learning (agentic RL). However, agentic RL still remains underexplored because of the bottlenecks from environmental interaction, reward calculation, and model training. In this work, we introduce AgentFly, an agentic RL framework that aims to mitigate these gaps by providing easy interfaces, maximal efficiency, and scalable training. AgentFly adopts a four-layer system design: (i) The agent layer exposes interfaces to users for customization. It abstracts the agent, tool, and reward concepts, and decomposes the agentic RL training into defining agents, tools, and reward functions; (ii) the rollout layer manipulates these components to form the agent loops and obtain rewards; (iii) the context layer organizes rollouts, injects contextual information and arrange resources; and (iv) the resource layer is responsible for low-level resource management, such as allocation and releasing. We also provide a suite of prebuilt tools and environments, demonstrating the framework's effectiveness through successful agent training across multiple tasks.[a]

[a] https://github.com/Agent-One-Lab/AgentFly

1 Introduction

Large language model (LLM) agents have been widely used in research and applications [19, 23, 26], where an LLM is taken as the core to interact with tools, environments, and APIs [12, 20]. Equipped with these external interfaces, LLMs are capable of completing tasks beyond their own capability like fetching online information. Typically, prompt engineering is used to build LLM agents, where all task information and available actions are given in prompts [23]. While training-based methods, such as supervised finetuning, improve LLMs by learning from agent interaction trajectories [2, 27]. Reinforcement learning offers a more powerful paradigm, as it directly optimizes long-horizon objectives and has been shown to substantially enhance LLMs reasoning, code, and other capabilities. However, employing RL for agentic settings has been shown to be challenging due to several issues.

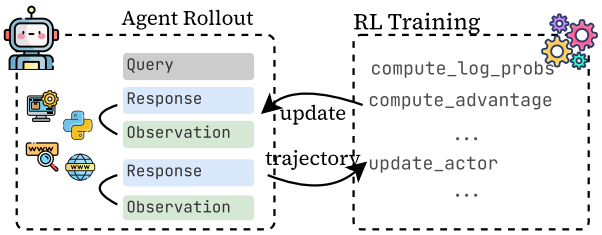

Figure 1: Decoupled agent rollout and training.

First, agentic RL naturally involves multi-turn interactions, where each turn consists of an LLM-generated action, followed by the resulting observation from the environment. Optimizing agents over such multi-turn trajectories remains underexplored, and their lengths pose scalability challenges for existing RL training infrastructures. Second, many RL algorithms, especially on-policy methods, require a rollout stage, where the

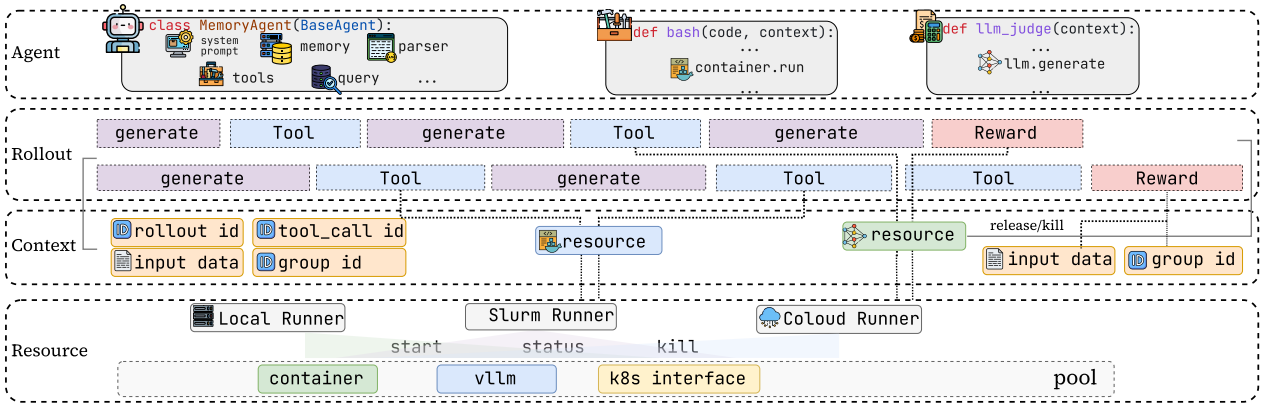

Figure 2: Overview of the structure of AgentFly framework. From top to bottom, we build agent, rollout, cotnext, and resource layers that orchestrate together to provide ease of use and maximal efficiency.

LLM generates multiple responses for different inputs to collect training trajectories. For LLM agents, rollout is particularly expensive: it involves not only text generation but also concurrent tool interactions. Efficiently handling this parallelism without sacrificing rollout throughput presents significant engineering challenges. Third, there is limited exploration of framework designs that balance modularity and RL training efficiency, leaving a gap in building systems that are both developer-friendly and scalable. Recent agentic RL systems implicitly or explicitly adopt a decoupled design between the agent and the training pipeline, as shown in Figure 1. However, there is still no consensus on the optimal architecture for agent systems designed for RL.

We propose AgentFly, a framework for training LLM agents with reinforcement learning. Our framework uses Verl [16] as RL trainer while building support for a more modular and scalable agentic RL setup. We made the following efforts to mitigate the above gaps:

1. We address multi-turn training by masking out tokens not generated by the LM when computing advantages and losses. This ensures that the model learns only from its own outputs, and we empirically validated this strategy's effectiveness in training LLM agents.
2. We unify agent-environment interaction by treating all interfaces as tools and transforming agent rollouts into an iterative tool-calling process. To support a simple and scalable tool calling and reward calculation, we designed an environment management system that maintains a dynamic pool of environment instances.
3. To promote usability and modular development, we adopt a four-layer design that supports easy agent, tool, and reward development while utilizing the flexibility of our framework to scale resources. Users can focus on defining agent workflows without needing to manage low-level training and resource management details.

In addition to the general framework design and implementation, we empirically validate AgentFly, by integrating four widely used RL algorithms and pre-building a suite of environments and tools. Using these components, we train LLM agents on six representative tasks, covering various interaction patterns and reasoning requirements. These experiments demonstrate the flexibility, scalability, and effectiveness of our framework in supporting diverse agentic RL training scenarios.

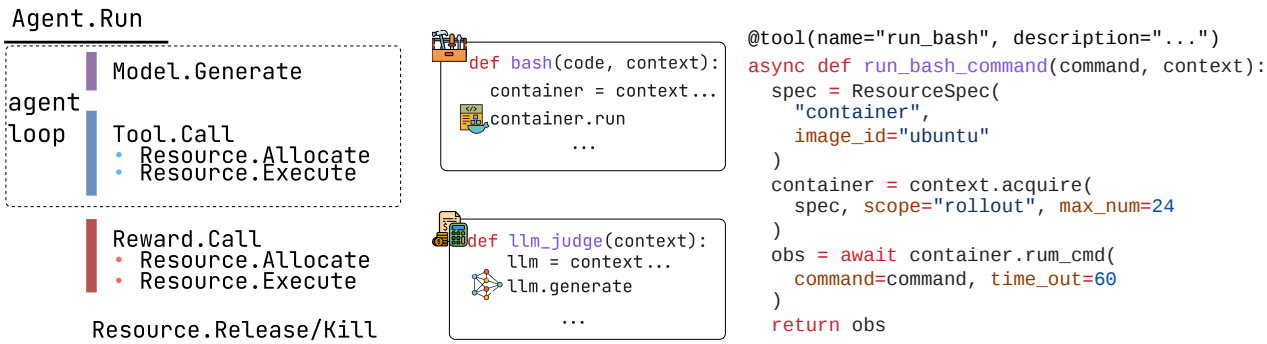

Figure 3: Left: Asynchronous structure of the system. Middle and right: code examples of tool and reward definitions.

2 AgentFly

2.1 High-Level Design Choices

Decoupled Agent Module. Reinforcement learning itself is systematic and consists of complex structures. To make agent development simple, we decouple the agent module from RL training. For the training process, we leverage `verl`, a widely-used open-source RL framework,[1] while for the agent part, we design a four-layer system to control the agentic workflow and return the trajectories to the training module. Figure 2 illustrates the overall structure of the framework.

Tool Abstraction. We use `tool` as an abstractive concept to represent all external interfaces the LLM agents interact with, including functions, APIs, environments, etc. Every agent interaction is treated as a tool invocation, and the corresponding external feedback is represented as a tool observation. This abstraction simplifies the integration and generalization of various interaction interfaces.

Fully Asynchronous Rollout. Agent rollout includes repeated token generation and tool calling, where the time cost for each step varies greatly. Multi-processing will block the generation to collect requests and launch processes, causing bubbles during rollout. To maximize efficiency, we adopt a fully asynchronous design for the entire rollout. As shown in Figure 3, token generation, tool calling, and reward calculation are implemented to be asynchronous. We also build an asynchronous resource system for resource scaling while not blocking the rollout.

2.2 Multi-Turn RL Training

Formally, we define an agent trajectory with k turns to be $T_k = (p, (r_1, o_1), (r_2, o_2), \ldots, (r_k))$, where p is the system and user prompt, r_i denotes the LLM-generated response, and o_i denotes the observations from tool outputs. For a single turn trajectory $T_1 = (p, (r_1))$, which is the case of RL for LMs, the PPO loss can be written as function w.r.t. current token a_t, its previous context s_t, and the advantage $\hat{A}_t$. :

$$\mathcal{L}_{PPO}(\theta) = \sum_{t=1}^{L} f(a_t, s_t, \hat{A}_t) * M_t$$

[1] https://github.com/volcengine/verl

Here M_t is the mask and $M_t = 1$ if $a_t \in r_1$, otherwise $M_t = 0$. To adapt the training to multi-turn, we directly make M_t into the multi-turn and apply it to the loss:

$$M_t = \begin{cases} 1 & \text{if } a_t \in \{r_1, r_2, \ldots, r_k\} \\ 0 & \text{otherwise} \end{cases}$$

In our implementation, we also apply masks to the advantage $\hat{A}$ to avoid unnecessary computations.

2.3 System Overview

We introduce the structure of our framework in this section, which consists of four layers to make agentic RL easy to use while maintaining efficiency.

2.3.1 Agent Layer

The agent layer exposes the main interfaces to users and consists of three main modules.

Agent. Users can define their own agents, which are the entities that complete the tasks, use tools, and do other operations like querying their memory. We have predefined some common interfaces to reduce the workload, like running the rollout, parsing tool calling, or getting trajectories. However, other custom agents can be added, such as a memory module.

Tool. We use tool to represent any external interface that the agent interacts with, including functions, APIs, and environments. Defining a tool is easy: by decorating the function or inheriting from the base class. However, agent tasks may be complex. An agent might be exploring within an operating system. This requires tools to be run in separate environments, while running asynchronously. To support such isolation and efficiency, while keeping tool definitions and usage simple, we use a context layer to collect information and communicate with the resource layer. An example that use context to acquire containers is shown in Figure 3.

Reward. To make the reward function both easy to define and flexible enough for complex evaluation, we adopt a similar design to that of the tools. A reward can take information or require resources from the context layer. For example, when the reward needs a code interpreter to validate the generated code, it can acquire a container from the context, then run the code inside the container.

2.3.2 Rollout Layer

The rollout layer is used to form the agent loop, calculating the reward and transferring rollout-related information to the context layer. Currently, we implement chain-like agent loops, where LLM-generated responses and tool observations are appended sequentially. However, the design of this layer makes it possible to extend to other types, like tree search. At the beginning or end of the rollout, it gives information like input data or rollout ids to the context layer, and may require resources from it.

2.3.3 Context Layer

The context layer is designed to inject information, communicate and transfer resources, and manage resource lifecycles, while making interfaces simple. For example, tools may acquire a container in their definition, or the reward may need a gold-standard answer for reference. During rollout, the context layer takes resources from the resource layer, gives it to tools, and injects gold-standard answer to rewards. After the rollout, it can kill resources if they are only required for that rollout.

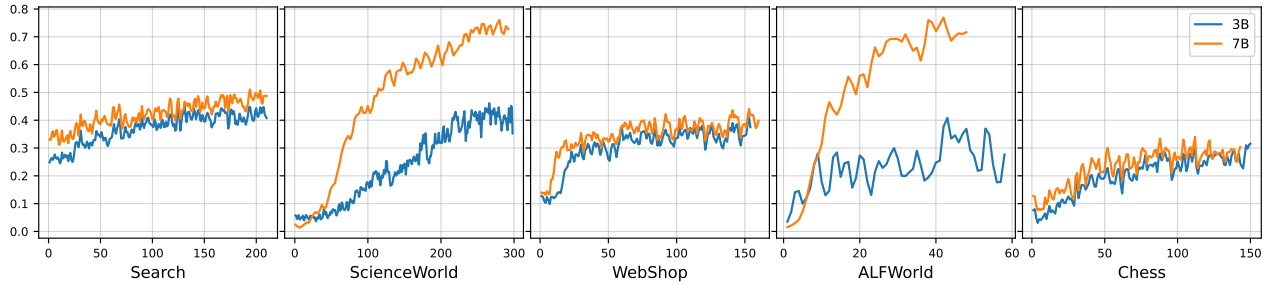

Figure 5: Reward curves for Qwen2.5-Instruct 3B and 7B models.

2.3.4 Resource Layer

To support large-scale and parallel usage of resources, we developed a resource management system for allocation and recycling. We define a resource unit to be the smallest single resource used by tools or rewards. Currently we have implemented container and vllm model as resource units, and runners to launch them in the form of local runners (which run resources on the local machine), and slurm runners (which use slurm to run resources on distributed systems). The system manages multiple resource pools, each containing a number of resource instances. After being acquired, the resource is taken from the pool, and upon release, it is returned for reuse. If no instance is available, the system queues the request until other rollouts free resources.

2.4 Prebuilt Tools / Environments

To illustrate the versatility of AgentFly, we have built some tools and environments: (I) We adopt the setting in Search-R1 [6] and design an asynchronous searching tool that retrieves documents from Wikipedia. (II) We adopt typical agent environments, ALFWorld [17], ScienceWorld [21], and WebShop [25], all of which are textual-based interactive environments. We wrap them into containers so that they can be scaled with our resource system. (III) We also designed a board game (Chess) environment and calculates reward using StockFish, so that the agent can learn to play board games. The tools and environments are described in detail in Appendix B.

3 Experiments

We detail our experiments in this section, where we first train with different RL algorithms for multi-turn agents, then present experimental results over different tools and environments.

3.1 Agentic RL with Different Algorithms

To evaluate the effectiveness of different reinforcement learning algorithms on AgentFly, we use a similar setting as Search-R1 [6], where the model retrieve passages from wikipedia to answer open-domain questions. We first evaluate four RL algorithms: PPO [14], REINFORCE++ [5], GRPO [15], and RLOO [1], all over Qwen2.5-Instruct 3B [13].

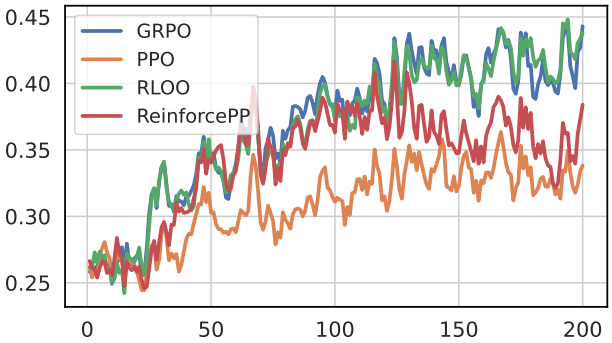

Figure 6: Reward curves of the four algorithms trained on the search tool to solve question answering tasks.

Algorithm	NQ	TriviaQA	HotpotQA	Musique
None	28.09	48.57	29.76	7.74
PPO	32.33	54.33	30.6	12.66
Reinforce++	37.7	56.46	27.48	5.96
GRPO	42.13	58.75	28.76	11.67

Table 1: Performance of question answering using search tool with different RL algorithms.

Algorithm	Dev		Creative		CAD		Sci		Office		OS	
	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon
QwenVL2.5-3B	16.2	1.4	23.3	1.4	10.2	4.7	38.2	6.4	24.3	3.8	15.0	1.1
QwenVL2.5-3B w/ GRPO	42.9	2.1	37.4	7.7	17.8	1.6	49.3	9.1	55.4	20.8	39.3	7.9
QwenVL2.5-7B	33.1	2.1	23.7	3.5	12.2	6.3	36.8	7.3	37.8	7.5	30.8	6.9
QwenVL2.5-7B w/ GRPO	46.8	3.4	39.7	8.3	19.6	5.1	52.8	10.7	58.6	22.7	42.8	9.3

Table 2: Performance of question answering using search tool with different RL algorithms.

The learning rate is set to 5×10^{-7}, and for each query, the model generates 5 trajectories. All training runs are conducted on NVIDIA H200 141GB nodes with 8 GPUs per node, and each full run takes approximately 200 GPU hours.

As presented in Figure 6, all algorithms exhibit similar reward trends, first increasing and then fluctuating. However, we found that different algorithms show different converging rewards. Quantitative results are shown in Table 1. GRPO performs best overall. We hypothesize this is due to the masking applied to the advantage function, which leads to more discrete token-level rewards and may hinder stable learning.

3.2 Agentic RL for Different Tools & Environments

Experimental Setup. We next conduct experiments on the tools and environments introduced in Section 2.4, using the same H200 cluster. We use GRPO [15] as the default algorithm for each task. Details of the hyperparameters and reward designs are provided in Appendix B.

As shown in Figure 5, both the 3B and 7B models can be effectively trained with agentic RL, showing clear reward improvements. In general, the 7B model achieves better performance and more significant gains compared to the 3B model. For complex tasks that require more turns and diverse tool usage, like ALFWorld and ScienceWorld, training progresses more slowly, as reflected by the gradually-increasing reward curves. But the final improvements are decent, showing that agentic RL is helpful for such long-turn tasks.

We also support multi-modal agents, and train Qwen2.5-VL on GUI tasks. To construct our training pipeline, we utilize the GUI-R1 [10] dataset. Furthermore, during the reinforcement learning phase, our reward calculation mechanism strictly follows the protocol established in UI-R1 [9]. Table 2 shows the performance on ScreenShotPro [8], significantly improving text-grounding capabilities across various domains despite the persisting challenge of purely visual icon grounding.

4 Analysis

4.1 How does the agent learning happen?

We explore how an LLM learns to complete a task in a multi-turn way by calling tools. Figure 7 shows how the number of tool calls and turns differs for the ALFWorld environment. For tasks in ALFWorld, the model generally needs to know available and valid actions or task objective as the start point. However, at the initial stage, the model simply tries to use the `step` tool to take actions. As the learning goes on, the model learns to use `admissible commands` and `task objective`, which reduce the need for trying different actions. In line with this, the average number of turns first increases and then decreases. We also find that the hallucination rate becomes less and less during learning, meaning that the model learns better format and tool consistency.

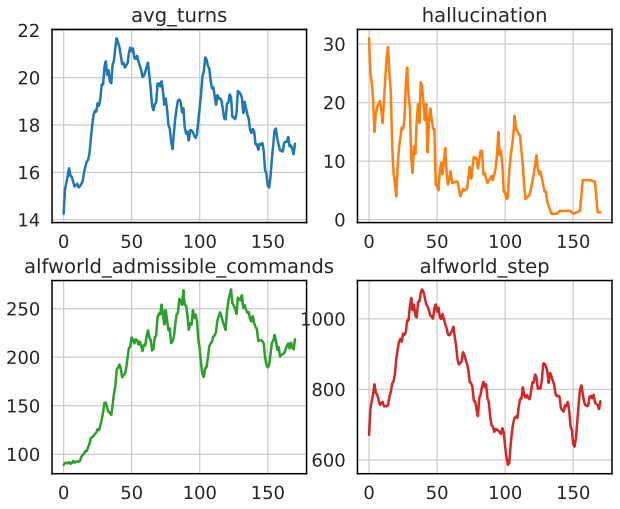

Figure 7: Statistics of number of tool calls, hallucinations and average turns for ALFWorld during training.

4.2 Tool Calling Format

During training, we have observed that using Qwen2.5's default tool calling format (i.e., JSON format) leads to failed agentic RL training. While there is a trend in using XML-based tool calling due to their simplicity for generation and parsing. And such properties are especially important for complex parameters like code, which may include quotes and brackets and cause LLMs to fail to generate a valid JSON string. We conduct experiments on the search task with XML and MCP formats. For XML, we use `<search>` and `</search>`, same as in Search-R1 [6].

Rewards and number of tool calls are shown in Figure 8. Rewards of both formats increase initially. However, as the training goes on, MCP's rewards downgrade while XML's rewards keep increasing and remain stable. The number of tool calls for MCP is also decreasing, showing that the model can not generate the correct tool calling format as the training goes on, which indicates the JSON format may not be appropriate for agentic RL.

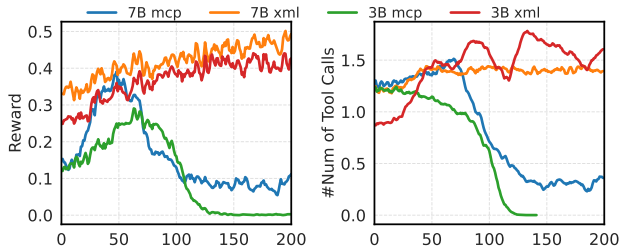

Figure 8: Comparison for reward curves and average number of tool calls for XML and MCP (JSON) format

4.3 Scaling Resources

We then scale the resources to show the effectiveness our resource system. As shown in Figure 3, we can configure the number of containers in the tool definition, and the context layer will acquire them from the low-level resource layer. We use the ScienceWorld task as an example. Except the resource number, other parameters are same as in Section 3. As shown in Figure 9, as the resource number increases, the time needed for training decreases and remains stable.

5 Related Work

There are several emerging RL training frameworks, some of which are mainly designed for LLMs and have agentic RL as an extension. Among them, `verl` [16] uses hybrid engines to train LLMs efficiently. AReal [4] orchestrates rollout and updating to be fully asynchronous with better efficiency, while suffering from the off-policy problem. Since such frameworks focus on training design and lack a systematic architecture for agentic RL, it's difficult to directly compare them.

We mainly compare with frameworks specifically designed for agentic RL. RAGEN [22] proposes StarPO in the form of a State-Thinking-Actions-Reward pipeline, but offers limited support for tool and environment extension and scaling. RL-Factory [18] features easy and efficient training, which integrates MCP-based tool definitions and asynchronous parallel tool-calls. Verl-agent [3] converts multi-turn agent trajectories into single-turn episodes through context appending, thereby converting multi-turn RL into multiple single-turn RL. They use ray to launch multiprocess environments, while making the rollout synchronous. Agent-Lightning [11] supports training with multiple agent front-ends, so migration cost is minimal. However, these front-ends are not designed for agentic RL, and suffer from isolation, efficiency, and scaling problems. Since different frameworks adopt different designs, we present a more systematic comparison in Table 3. Overall, ours show a balance between ease of development and efficiency.

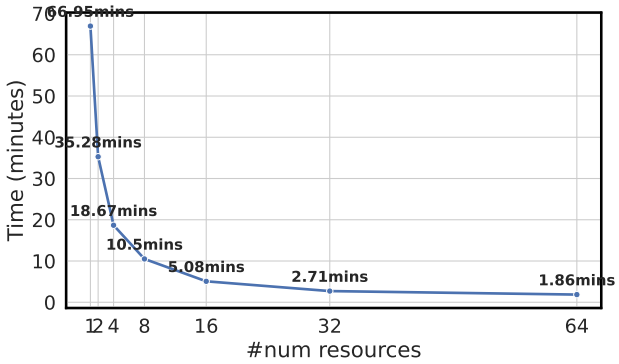

Figure 9: Time spent per step as the number of resources increases.

6 Conclusion

We propose AgentFly, an easily-extensible and scalable agent reinforcement learning framework. The framework supports multi-turn RL training by applying masks. To support scalable training, we designed an asynchronous tool and reward system, a central resource system to manage the scaling of environments by simply increasing the environment instances. We decoupled the agent module to make further development simple. We additionally built several tools and experimented on models of different sizes. We found that even a 3B model can be used in complex tasks requiring multiple turns, while the training becomes more unstable with more turns. Generally, larger models achieve better performance.

References

[1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.662. URL https://aclanthology.org/2024.acl-long.662/.

[2] Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. FireAct: Toward language agent fine-tuning, 2023.

[3] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for LLM agent training. *arXiv preprint arXiv:2505.10978*, 2025.

[4] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL https://arxiv.org/abs/2505.24298.

[5] Jian Hu, Jason Klein Liu, and Wei Shen. Reinforce++: An efficient RLHF algorithm with robustness to both prompt and reward models. *URL https://arxiv. org/abs/2501*, 3262:32–33, 2025.

[6] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

[7] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

[8] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 8778–8786, 2025.

[9] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanjing Xiong, and Hongsheng Li. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. *arXiv preprint arXiv:2503.21620*, 2025.

[10] Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025.

[11] Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K Qiu, and Yuqing Yang. Agent lightning: Train any ai agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025.

[12] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024.

[13] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL https://arxiv.org/abs/2412.15115.

[14] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[15] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[16] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv: 2409.19256*, 2024.

[17] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew J. Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. *CoRR*, abs/2010.03768, 2021. URL https://arxiv.org/abs/2010.03768. Presented at the 9th International Conference on Learning Representations (ICLR 2021).

[18] Simple-Efficient. Rl-factory. https://github.com/Simple-Efficient/RL-Factory, 2025.

[19] Theodore Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2023.

[20] Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. ToolGen: Unified tool retrieval and calling via generation. In *The Thirteenth International Conference on Learning Representations*, 2025.

[21] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? *arXiv preprint arXiv:2203.07540*, 2022. URL https://arxiv.org/abs/2203.07540.

[22] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning, 2025. URL https://arxiv.org/abs/2504.20073.

[23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.

[24] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1259. URL https://aclanthology.org/D18-1259/.

[25] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. *arXiv preprint arXiv:2207.01206*, 2022. URL https://arxiv.org/abs/2207.01206.

[26] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[27] Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. AgentTuning: Enabling generalized agent abilities for LLMs. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3053–3077, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.181. URL https://aclanthology.org/2024.findings-acl.181/.

Framework	Agent Abstraction	Async	Resource Scaling	Multi-Modal	Backends
AgentFly	Yes, with agent, tool, and reward abstraction	Yes	Dedicated resource system	Text and image	verl
RAGEN	No	No	No, uses ray	No	verl
verl-agent	No	No	Uses ray with multi processing	Qwen3-VL support	verl
RL-Factory	No, with tool config plus reward functions	Yes	No direct support, depends on how tool is designed	Text and image	verl
agent-lightning	N/A, depends on the agent framework used	Yes	No direct support, depends on how tool is designed	N/A; depends on agent framework used	LangChain, AutoGen, CrewAI, OpenAI SDK for agent rollout
verl-tool	No, only tool abstraction	Yes	No direct support, depends on how tool is designed	Text, image and video	verl
SkyRL-Agent	Yes, with tool and agent abstraction	Yes	Yes, uses K8S	No	SkyRL-train, verl, Tinker

Table 3: Comparison of different agentic RL frameworks.

A Agentic RL Framework Comparison

We provide a multi-dimensional comparison for different agentic RL frameworks in Table 3.

B Tool Implementations

B.1 Search

For the search tool, we use the same data and model as in Search-R1 [6], and search asynchronously.

We randomly sampled examples from HotpotQA [24] and Natural Questions [7] training set as the training data, and directly use the exact match score of the final response and the gold-standard answer as the reward.

B.2 ALFWorld Implementation Details

Our integration with the ALFWorld [17] is built upon a decoupled, service-oriented architecture to ensure stability and scalability. We treat the complex, stateful ALFWorld simulation as an "Environment-as-a-Service," which runs isolated in a **container** with a **FastAPI** web server. Our framework interacts with this service via an asynchronous HTTP client, the `ALFWorldEnv` class.

The reward signal follows the same robust principle of relying on the environment as the single source of truth. Manually scripting rewards for a complex task like ALFWorld is tricky. Instead, we leverage our direct

line to the simulation server to get an **outcome-based reward**. Our framework's reward function simply polls the environment. The server checks if the task's final goal conditions have been met, and returns the definitive sparse reward (1.0 for success, 0.0 otherwise). This ensures the agent is optimized for the true task objective.

Agent Reasoning and the Action Space.. Our agent leverages the environment's built-in support for admissible commands. At any point, the agent can use the `alfworld_get_admissible_commands` tool to query the environment for a list of all currently valid and contextually relevant actions.

B.3 Webshop Implementation Details

We integrate with Webshop environment [25] by adapting its text-only version and running commands in an isolated containerized **FastAPI** web server, using an overall client-server interaction. The agent then interacts with the service via an asynchronous HTTP client. We follow the standard reward signal used in the WebShop environment, using programmatic functions that match the attributes, types, options, and price between the desired product and purchased product.

We guide the agent by adding environment info and description in the tool definition, both appended with the system prompt. We observe that the instruction model is capable of following instructions, and we see lower numbers of invalid tool calls as the training progresses. The agent interacts with the environment as if it were browsing through a shopping website. There are two admissible actions for the agent to navigate through the environment: `search` and `click`.

B.4 ScienceWorld Implementation Details

We wrap the original ScienceWorld environment implementation [21] as a containerized FastAPI server, which is accessible via an asynchronous HTTP client; Each task in ScienceWorld consists of 2 to 15 sub-goals, which yield partial rewards; completing the full task earns the maximum reward (1). In our multi-turn framework, we define the task reward as the highest reward achieved at any point along the trajectory of the model.

ScienceWorld has a vast number of possible action spaces. However, unlike the approach we use for ALFWorld, we only provide the list of admissible commands in the system prompt and tool definition. While the agent conducts invalid actions a number of times, we observe that it can still take valid actions with the given system prompt and achieve a higher average reward as the training progresses.

B.5 Chess Implementation Details

Unlike the containerized environments used for ALFWorld, WebShop, and ScienceWorld, the chess puzzle environment runs using the `python-chess` library for board state management and move validation, along with the **Stockfish** engine for position evaluation. Puzzles are drawn from the Lichess database in a standard format: each puzzle specifies a FEN string (the starting position) and a sequence of UCI solution moves. The first move in the sequence is the opponent's "setup" move, which is auto-played by the environment; the agent must then find all subsequent moves to solve the puzzle.

Tools.. The agent interacts with the chess environment through three tools, all declared as stateful with `pool_size=8`:

- `chess_get_state`: returns the current FEN, an ASCII board visualization, whose turn it is, the number of legal moves, check/checkmate/stalemate status, and the list of moves played so far.
- `chess_get_legal_moves`: returns a sorted list of all legal moves, each displayed in `"uci (san)"` format (e.g., `e2e4 (e4)`).

- `chess_move(move)`: accepts a move in UCI or SAN notation, validates legality, checks it against the puzzle solution, auto-plays the opponent's response if the move is correct, and returns result text plus the updated board state.

Reward Design.. We provide two reward functions for chess puzzle training:

- **Dense reward**: combines a solve component and a move-quality component as $r = 0.6 \times r_{\text{solve}} + 0.4 \times r_{\text{quality}}$. The solve reward is 1.0 if the puzzle is fully solved, or partial credit up to 0.5 proportional to the fraction of solution moves correctly played. The move-quality reward evaluates each agent move against Stockfish's best move: if the agent's move matches the engine's recommendation it scores 100; otherwise the centipawn loss is computed and normalized to a 0–100 scale (with -300 cp loss mapping to 0). The per-move scores are averaged and scaled to $[0, 1]$.
- **Binary reward**: returns 1.0 if the puzzle is solved, 0.0 otherwise.